

HOSTEL MANAGEMENT SYSTEM

Product Requirements Document + Diagram Specification

Version 1.0 | Master Design Specification | September 2026

Real-Time • Secure • Role-Based • Responsive • End-to-End

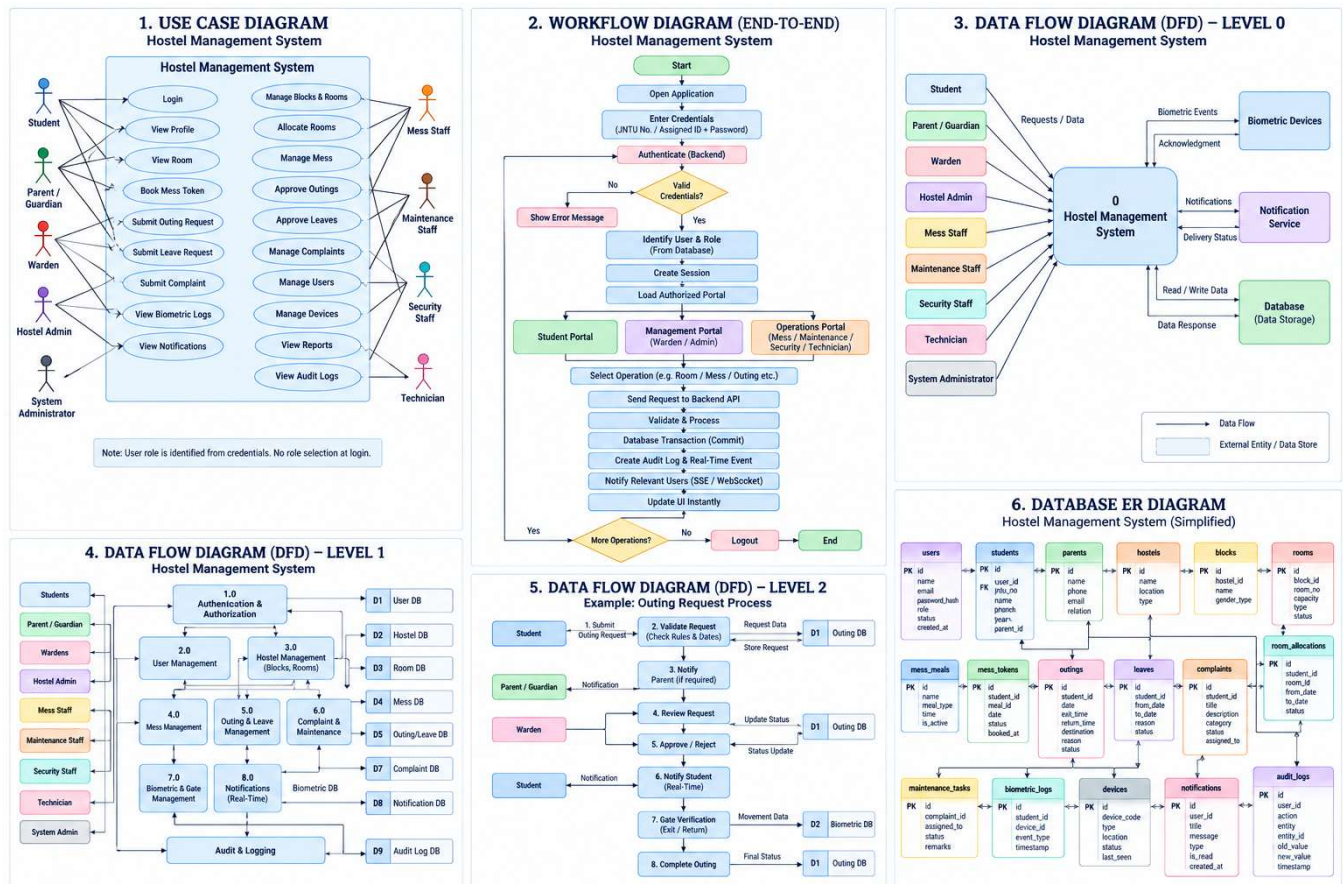


Figure 1. Master visual blueprint: use cases, workflow, DFD levels and database model.

1. Document Control

Item	Specification
Document	Hostel Management System — PRD and Diagram Specification
Version	1.0
Status	Master design specification
Product type	Real-time responsive web application
Authentication	Credentials only; no role-selection dropdown
Initial delivery	Standalone hostel application
Future integration	Integration with the existing CMS after standalone completion

2. Product Vision

Build a centralized hostel platform that manages student accommodation and hostel operations through secure APIs, a shared authoritative database, real-time event propagation, and role-based interfaces. The first implementation remains standalone; CMS integration is deferred until the hostel product is complete and verified.

3. Core Product Principles

- No role-selection dropdown at login. The backend determines the account role from the issued credentials.
- The database is the authoritative source for persistent business data.
- Every state-changing operation is authenticated, authorized, validated and persisted transactionally.
- Real-time events update connected clients without requiring manual page refresh.
- Reconnection must resynchronize client state so temporary network loss does not create stale data.
- The UI must be responsive on mobile, tablet and desktop.
- No hardcoded production credentials, student records, operational states or mock business data.
- Audit/history is retained for important state-changing operations.

4. Actors and Responsibilities

Actor	Primary responsibilities
Student	Room, mess tokens, outings, leaves, complaints, biometric history, notifications
Parent / Guardian	Permitted student-linked approvals, information and notifications
Warden	Hostel operations, request approvals, complaints, monitoring
Hostel Admin	Blocks, rooms, allocations, configuration, users, reports
Mess Staff	Meals, tokens/indents, attendance/usage and mess operations
Maintenance Staff	Complaint assignment, work progress, resolution and closure
Security / Gate Staff	Authorized entry/exit verification and movement records
Technician	Device registration, health, maintenance and device activity
System Administrator	Users, roles, permissions, configuration, audit and security

5. Authentication and Authorization

Authentication is credential-driven. Users do not select their role.

Credentials issued by institution



Backend authentication
↓
Account lookup + password verification
↓
Role resolved from authoritative account record
↓
Authenticated session
↓
Authorized portal / permissions

- Student login: JNTU No. + password.
- Staff/admin accounts use their assigned credentials.
- Role must be resolved server-side and must never be trusted from a client-supplied role field.
- Use secure password hashing and protected sessions/tokens.
- Implement logout, session expiry, login rate limiting and generic invalid-credential errors.
- UI visibility is not authorization; every protected API must enforce authorization server-side.

6. Functional Scope

Portal	Modules
Student	Dashboard; My Room; Mess Tokens; Outing Requests; Complaints; Leaves & Suspension; Biometric Tracking; Notifications
Management	Dashboard; Block Management; Room Allocation; Mess Management; Outing Approvals; Maintenance; Log History; Outing Log History; User Management; Guest Billing; Leaves & Suspension; Device Management; Notifications

7. Functional Requirements

7.1 Student Dashboard

- Show personalized student identity, room/stay, active outings, mess status and leave status.
- Provide quick actions to Outing Request, Biometric, Complaint and Mess Tokens.
- All values must be derived from backend data.

7.2 Block and Room Management

- Create, view, edit, search, filter and deactivate blocks and rooms.
- Maintain Block → Floor → Room → Student allocation relationships.
- Prevent over-allocation, duplicate active allocations and invalid references.
- Use transactions for allocation changes.

7.3 My Room

- Show block, floor, room number, room type, capacity, occupancy and permitted roommate information.
- Reflect allocation changes from the authoritative backend.

7.4 Mess Management

- Configure meals, timings, availability and booking deadlines.
- Allow students to view meals and book/cancel tokens where policy permits.
- Backend enforces deadlines, duplicate booking rules and availability.

7.5 Outing Requests

- Student creates request with required date/time, destination and reason details.
- Request follows configured approval workflow.
- Approved outings support authorized gate exit and return records.
- Maintain complete outing history.

7.6 Leaves and Suspension

- Students submit leave requests with type, reason and dates.
- Validate dates and configured policy rules.
- Maintain approval, active and completed states separately from suspension records.

7.7 Complaints and Maintenance

- Students create complaints with category, description and optional images.
- Management assigns work to maintenance staff.
- Track Created → Assigned → In Progress → Resolved → Closed.
- Students receive status updates.

7.8 Biometric Tracking

- Accept authenticated device events.
- Validate device, student, event type and timestamp.
- Store authoritative movement records.
- Provide permitted logs and analytics to students/staff.

7.9 Notifications

- Support announcements, reminders, events and operational notifications.
- Track recipient, type, message, timestamp and read/unread state.
- Unread count is derived from actual notification records.

7.10 User and Device Management

- Authorized administrators manage accounts, roles, permissions and device records.
- Device secrets are handled securely and are not exposed unnecessarily.

7.11 Guest Billing

- Track guest, host association, visit, billing items, amount, payment status and references.
- Financial mutations must be transactionally consistent.

7.12 Audit and History

- Record actor, action, entity, entity ID, timestamp, state changes and metadata for important mutations.
- Historical records must not be silently overwritten.

8. Real-Time Architecture Requirements

Real-time behavior is a mandatory product capability. The recommended implementation uses REST APIs for commands/initial state and SSE or WebSocket for server-to-client event propagation.

UI action
↓
API request

```

↓
Authentication → Authorization → Validation → Business rules
↓
Database transaction
↓
Commit
↓
Audit / notification / domain event
↓
SSE / WebSocket
↓
Connected clients update

```

- The database commit is authoritative; real-time events must not precede an uncommitted state change.
- Clients should fetch initial state via API and then subscribe to real-time updates.
- On reconnect, the client must resynchronize authoritative state.
- Events should be identifiable so duplicate delivery can be handled safely.
- Authentication expiry must be handled without leaving a stale authenticated UI.

9. End-to-End Master Workflow

```

START
↓
Open Application
↓
Enter Credentials
↓
Backend Authentication
├─ Fail → Error → Retry
└─ Pass → Identify Account + Role
    ↓
    Create Session
    ↓
    Load Authorized Portal
    ↓
    Select Operation
    ↓
    API Request
    ↓
    Authentication → Authorization → Validation → Business Rules
    ↓
    Database Transaction
    ├─ Fail → Rollback → Error
    └─ Success → Commit
        ↓
        Audit / Notification
        ↓
        Real-Time Event
        ↓
        Client UI Update
        ↓
        Continue
        ↓
        Logout
        ↓
        END

```

10. Critical Business Workflows

Workflow	End-to-end sequence
Room Allocation	Admin/Warden → Select Student → Select Available Room → Validate Capacity/Existing Allocation → Transaction → Allocation Created → Audit → Real-Time Student Update
Mess Token	Student → Select Date/M meal → Check Availability/Deadline/Duplicate → Transaction → Token Created → Notification/Event → UI Updated
Outing	Student → Create Request → Validate → Required Approval(s) → Warden Review → Approve/Reject → Notify → If Approved: Gate Verification → Exit → Return → Complete → History
Leave	Student → Create Leave → Validate Dates/Policy → Approval → Approved/Rejected → Active → End Date → Completed → History
Complaint	Student → Create Complaint → Validate/Upload → Queue → Assign → In Progress → Resolve → Notify Student → Close
Biometric	Device → Authenticate Device → Validate Event → Store Movement Record → Audit → Real-Time Event → Authorized Dashboards/Analytics

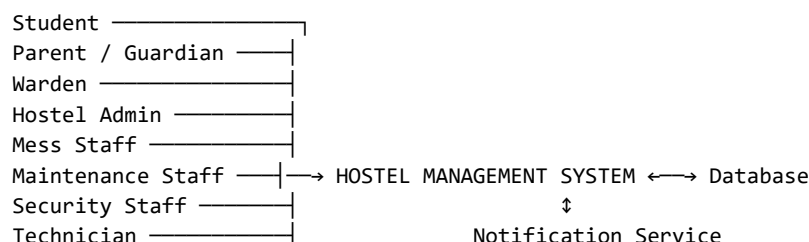
11. Use Case Specification

Actor	Use cases
Student	Login; View Dashboard; View Room; View Roommates; Book Mess Token; Submit/View Outing; Submit/View Leave; Submit/View Complaint; View Biometric; View Notifications; Logout
Parent/Guardian	Login; View linked permitted information; Participate in eligible approval; Notifications
Warden	Dashboard; Room/occupancy view; Outing approval; Leave decisions; Complaint monitoring; Movement monitoring; History
Hostel Admin	Blocks; Rooms; Allocations; Users; Configuration; Mess configuration; Reports; Notifications
Mess Staff	Meals; Token operations; Attendance/usage; Statistics
Maintenance Staff	Assignments; Status updates; Resolution; Closure
Security Staff	Authorized outing validation; Gate exit; Gate return; Movement history
Technician	Devices; Health; Configuration; Maintenance; Activity
System Admin	Users; Roles; Permissions; Configuration; Audit; Security

12. DFD Specification

12.1 DFD Level 0 — Context

The system is represented as one process: Hostel Management System. External entities exchange requests, operational data and responses with it.



System Administrator └



Biometric / Gate Devices

12.2 DFD Level 1 — Major Processes

Process	Responsibility	Primary data stores
1.0 Authentication & Authorization	Credentials, session and role resolution	User DB
2.0 User Management	Accounts, roles, permissions	User DB
3.0 Hostel Management	Blocks, floors, rooms, allocations	Hostel/Room DB
4.0 Mess Management	Meals, tokens, bookings	Mess DB
5.0 Outing & Leave Management	Requests, approvals, gate lifecycle	Outing/Leave DB
6.0 Complaint & Maintenance	Complaints, assignments, resolutions	Complaint DB
7.0 Biometric & Gate Management	Movement and device events	Biometric/Device DB
8.0 Notifications	Create/deliver/read notifications	Notification DB
9.0 Audit & Logging	Immutable operational history	Audit DB

12.3 DFD Level 2 — Outing Request Example

Student



2.1 Submit Outing Request



2.2 Validate Request (identity, dates, policy)



2.3 Create Outing Record



2.4 Notify Parent/Warden where required



2.5 Warden Approve / Reject



Update Outing Status + Audit



2.6 Notify Student (Real-Time)



2.7 Gate Verification (Exit / Return)



2.8 Complete Outing + History

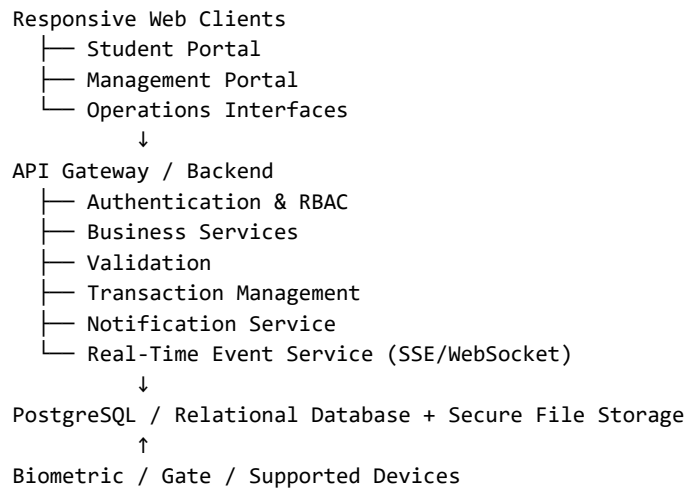
13. Database / ER Model Specification

Entity	Key relationships / purpose
User	Authentication, role, account status
Student	Student profile linked to User; JNTU number
Parent/Guardian	Linked to permitted student records
Hostel	Top-level hostel configuration
Block	Belongs to hostel
Room	Belongs to block/floor; capacity/status
RoomAllocation	Student ↔ Room history and active allocation
Meal	Meal type, timing and configuration
MessToken	Student ↔ Meal booking/attendance
OutingRequest	Student request, approval and gate lifecycle
LeaveRequest	Student leave workflow
Suspension	Separate disciplinary/administrative state
Complaint	Student issue and lifecycle
MaintenanceTask	Complaint assignment and resolution
BiometricDevice	Registered movement device
BiometricLog	Authenticated movement event
Notification	Recipient and notification state
Guest/Bill	Guest visit and billing records

AuditLog	Actor, action, entity, before/after state and timestamp
----------	---

User 1 — 1 Student
 Student 1 — N RoomAllocation N — 1 Room
 Hostel 1 — N Block 1 — N Room
 Student 1 — N MessToken N — 1 Meal
 Student 1 — N OutingRequest
 Student 1 — N LeaveRequest
 Student 1 — N Complaint 1 — N MaintenanceTask
 Student 1 — N BiometricLog N — 1 BiometricDevice
 User 1 — N Notification
 User 1 — N AuditLog

14. System Architecture Specification



15. Responsive UI Specification

Viewport	Requirements
Mobile: 320px–639px	Mobile-first; drawer navigation; touch-friendly controls; single-column layouts; no horizontal overflow
Tablet: 640px–1023px	Adaptive grids; controlled content width; touch and mouse support
Desktop: ≥1024px	Sidebar navigation; multi-column dashboards; tables/data grids; expanded analytics
Large desktop: ≥1440px	Constrained content widths; balanced whitespace; no excessive stretching

Use shared components with responsive CSS/Grid/Flexbox. Do not create separate mobile and desktop applications unless a genuine interaction difference requires it.

16. Non-Functional Requirements

- Security: authentication, authorization, input validation, secure file handling, rate limiting and secret management.
- Reliability: transactional updates, error recovery, real-time reconnection and state resynchronization.
- Performance: indexed database queries, pagination for large lists and efficient real-time subscriptions.
- Accessibility: semantic labels, keyboard navigation, focus states, readable errors and adequate touch targets.

- Observability: structured server logs, audit records, health checks and actionable error reporting.
- Maintainability: modular services/components, migrations, documented APIs and consistent validation.
- Scalability: avoid coupling real-time delivery to a single browser session or process-local state where multi-instance deployment is expected.

17. End-to-End Test Matrix

Flow	Minimum verification
Authentication	Valid login; invalid login; disabled account; logout; session expiry; rate limiting
Room	Create room; allocate; capacity enforcement; reallocation; student sees updated room
Mess	View meals; booking; deadline enforcement; duplicate prevention; cancellation if allowed
Outing	Submit; approval/rejection; real-time notification; gate exit; gate return; completion
Leave	Submit; date validation; approval/rejection; active state; completion
Complaint	Create; attachment validation; assignment; progress; resolution; closure
Biometric	Device authentication; event validation; persistence; duplicate/replay handling; analytics
Notifications	Creation; delivery; unread count; read state; real-time update
Authorization	Cross-role denial; object-level access denial; protected routes/API endpoints
Responsive	360, 390, 412, 768, 1024, 1366 and 1920 width verification

18. Implementation Phases

- Phase 1 — Requirements and diagrams
- Phase 2 — Database and backend foundation
- Phase 3 — Authentication and RBAC
- Phase 4 — Management Portal
- Phase 5 — Student Portal
- Phase 6 — Real-time event infrastructure
- Phase 7 — Device/gate integration
- Phase 8 — Notifications and audit
- Phase 9 — End-to-end testing
- Phase 10 — Production hardening
- Phase 11 — Integration into the existing CMS

19. Definition of Done

- All specified modules have working backend persistence and frontend flows.
- No production dependency on mock arrays or hardcoded operational records.
- No role-selection dropdown at login; role is determined from authenticated account data.
- RBAC is enforced on the server.
- Critical workflows are transactional and audited.
- Real-time events update relevant clients and reconnect safely.
- Mobile and desktop layouts are verified.

- Unit, integration, security and E2E tests are passing for the implemented scope.
- Production build succeeds and deployment configuration is documented.
- Only after standalone verification is complete should CMS integration begin.

20. Diagram Deliverables

Diagram	Purpose	Status
Use Case Diagram	Actors and permitted system operations	Defined
Master Workflow	End-to-end operational sequence	Defined
DFD Level 0	System context and external data exchanges	Defined
DFD Level 1	Major processes and data stores	Defined
DFD Level 2	Detailed critical process, e.g. outing	Defined
ER Diagram	Entities and relationships	Defined at simplified level
System Architecture	Client/API/DB/real-time/device relationships	Defined

21. Implementation Guardrails

- Inspect the existing project before introducing dependencies or replacing architecture.
- Implement one module at a time and verify it before starting the next.
- Do not claim functionality is complete without testing the real flow.
- Do not integrate with the CMS until the standalone hostel application is functionally verified.
- Treat this document as the current source of truth; future changes should be versioned rather than silently diverging.

END OF PRD v1.0